

Drive Portal - Role Based File Management System

Project Overview

Drive Portal is a role-based file management system developed using React, Node.js, Express.js, and PostgreSQL.

The application allows users to upload, manage, organize, search, preview, download, and delete files through a secure dashboard. It supports role-based access control where administrators can manage all files while normal users can access only their own files.

Features

Authentication

- User Registration
- User Login
- Password Encryption using BCrypt
- Role-Based Access Control (Admin/User)

File Management

- Upload Files
- Download Files
- Delete Files
- Rename Files
- Preview Images and PDF Files
- Search Files

Folder Management

- Create Folder
- Rename Folder
- Delete Folder
- Nested Folder Support
- Move Files Between Folders

Role-Based Access

Admin

- View all uploaded files
- View file owners
- Manage all files
- Create folders
- Rename folders

- Delete folders

User

- View only own files
 - Upload files
 - Download files
 - Rename own files
 - Delete own files
 - Search own files
-

Technology Stack

Frontend

- React.js
- React Router DOM
- Axios
- React Dropzone
- React Icons
- CSS

Backend

- Node.js
- Express.js
- Multer
- BCrypt
- CORS

Database

- PostgreSQL

File Storage

Files are stored in a local uploads directory as specified in the project requirements.

Project Structure

DrivePortal/

├── frontend/

| ├── src/

| ├── pages/

| └── components/

```
|   └── assets/

|

|── backend/

|   ├── uploads/

|   ├── server.js

|   └── db.js

|

|── screenshots/

|── database_schema.sql

|── README.md

└── postman_collection.json
```

Database Schema

Users Table

Stores user account information.

Fields:

- id
- username
- email
- password
- role

Files Table

Stores uploaded file information.

Fields:

- id
- filename
- filesize
- filetype
- userid
- folder_id

- uploaded_at

Folders Table

Stores folder information.

Fields:

- id
 - name
 - parent_id
-

Installation

Clone Repository

```
git clone <repository-url>
```

Backend Setup

```
cd backend
```

```
npm install
```

```
npm start
```

Frontend Setup

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Environment

Backend Server:

```
http://localhost:5000
```

Frontend Server:

```
http://localhost:5173
```

API Endpoints

Authentication

POST /register

POST /login

File Management

POST /upload

GET /files/:userId/:role

DELETE /file/:id

PUT /rename/:id

GET /download/:filename

GET /search/:term/:userId/:role

Folder Management

POST /folder

GET /folders

GET /folder/:id

DELETE /folder/:id

PUT /rename/folder/:id

GET /folder-files/:id/:userId/:role

GET /subfolders/:id

GET /breadcrumb/:id

File Operations

PUT /move-file/:id

Security Features

- Password Hashing using BCrypt
- Role-Based Authorization

- User Specific File Access
 - Admin Privilege Validation
-

Future Improvements

- Cloudinary Integration
 - AWS S3 Storage
 - JWT Authentication
 - File Sharing via Links
 - Email Notifications
 - Activity Logs
 - Dark Mode
-

Screenshots

Include screenshots of:

1. Login Page
 2. Registration Page
 3. Dashboard
 4. File Upload
 5. Folder Management
 6. Search Functionality
-

Author

Thenmozhi M

B.E Computer Science and Engineering

Role Based File Management System Internship Project